

Summary

Floating-point representation is a structural solution to the fact that computers cannot store a physical decimal point. Instead of storing a dot, computers store where the dot should be, using a structured format defined by the IEEE 754 standard.

By decomposing a number into a Sign bit, a Biased Exponent, and a Mantissa, IEEE 754 enables representation of extremely small fractions and extremely large numbers while maintaining high hardware efficiency.

Key concepts include normalization, exponent biasing, and recognition of special values such as Zero, Infinity, NaN, and Denormalized numbers.

I. Fixed Point vs. Floating Point Representation

Fixed-Point Numbers

- **Definition:** Numbers with a fixed and predetermined position for the decimal point
- **Storage Logic:**
 - The decimal point is **not stored**
 - Its position is assumed by hardware
- **Operational Advantage:**
 - Arithmetic operations are identical to integer operations
 - Hardware ignores the point during computation
- **Limitations:**
 - Cannot efficiently represent very large or very small numbers
 - Requires large bit-width for flexibility

Floating-Point Numbers

- **Definition:** Numbers where the decimal point can appear at variable positions
- **Necessity:**
 - Computers store only 0s and 1s using flip-flops
 - A physical decimal point cannot be stored
- **Core Challenge:**
 - Removing the point directly destroys the number's value
 - Example:
 - 23.5 and 0.938 become meaningless if the point is removed
- **Solution:**
 - Store the **position of the point**, not the point itself

II. Normalization Process

Normalization is a mandatory step before storing a floating-point number.

Rule of Normalization

A number is normalized when exactly one non-zero digit exists to the left of the decimal point.

- **Binary Constraint:**
 - Only one non-zero digit exists: 1
- **Normalized Binary Format:**
 $1.f \times 2^e$
- **Implicit Leading Bit:**
 - Since the leading digit is always 1, it is not stored
 - This saves storage space
 - Known as the Implicit Format

III. IEEE 754 Standard Architecture

IEEE 754 defines the internal structure for floating-point storage in two main formats.

Format Comparison

Feature	Single Precision (Float)	Double Precision (Double)
Total Width	32 bits	64 bits
Sign Bit	1 bit	1 bit
Exponent	8 bits	11 bits
Mantissa	23 bits	52 bits
Bias Value	127	1023

Core Components

1. **Sign Bit (S)**
 - 0 → Positive
 - 1 → Negative
2. **Exponent (E)**
 - Stores the position of the decimal point
3. **Mantissa (M)**

- Stores the fractional part of the normalized number
- Represents digits following the leading 1

IV. Exponent Biasing Logic

IEEE 754 does not store the true exponent. Instead, it stores a biased exponent.

Why Biasing Is Used

- Floating-point arithmetic requires exponent comparison
- If exponents were signed:
 - Hardware would need to check sign and magnitude
 - This slows computation
- Biasing converts all exponents into positive numbers
- Allows unsigned integer comparison, increasing speed

Bias Calculation Formula

$$\text{Bias} = 2^{(k-1)} - 1$$

(where k = number of exponent bits)

Example: Single Precision

- k = 8
- Bias = $2^7 - 1 = 127$
- Allows representation of true exponents approximately from -126 to +127

V. Special Values and Extreme Cases in IEEE 754

Certain exponent patterns are reserved for exceptional conditions.

Case	Exponent Bits	Mantissa Bits	Meaning
Normal Number	Not all 0s or 1s	Any	Valid normalized number
Zero	All 0s	All 0s	Underflow; cannot be normalized
Denormalized	All 0s	Non-zero	Smaller than smallest normalized value (Underflow)

Infinity	All 1s	All 0s	Overflow, often from division by zero
NaN	All 1s	Non-zero	Undefined or invalid result (Overflow)

VI. Competitive Examination Strategy (GATE Focus)

Conceptual Mastery over Computation

- Average students focus on solving numerical problems
- High performers master concepts, making computation trivial
- Memorize binary weights instead of repeated division
 - Example:
 - $10000010_2 = 128 + 2 = 130$

Time Management Strategy

- Avoid treating hard questions as personal challenges
- If no progress in 5–7 minutes, the question is likely rank-filtering
- Skipping strategically preserves time and confidence

Product Calculation Risk Awareness

- Floating-point multiplication involves:
 - Adding exponents
 - Multiplying mantissas
- If resulting exponent becomes all 1s:
 - Output becomes Infinity or NaN
 - Mantissa precision becomes irrelevant
- Early identification prevents unnecessary computation